Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-208БВ-25

Студент: Юсупов Юсуп

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 20.10.26

Москва, 2026

Постановка задачи

Вариант 5.

Необходимо составить программу процесса, принимающего на вход название файла, который будет обрабатывать команды типа: “Число<endline>”. Затем процесс должен создать дочерний процесс, которому будет отправлено данное число. Если число окажется составным, наследник должен записать его в файл, иначе — родительский и дочерний процессы завершаются.

Общий метод и алгоритм решения

Программа запускает дочерний процесс, с помощью которого обрабатываются вводимые пользователем данные, которые записываются в файл, если данные — составные числа.

Использованные системные вызовы:

- pid_t fork(); – создает дочерний процесс.
- int pipe(int arr[]); – создает канал связи между процессами.
- int read(int fd, void *buf, size_t count); – низкоуровневое чтение через файловый дескриптор.
- int write(int fd.-, const void *buf, size_t count); – низкоуровневая запись через файловый дескриптор.
- int dup(int fd); – создает файлового дескриптора, ссылающегося на тот же файл, что и fd.
- int dup2(int old_fd, new_fd); – направляет new_fd туда же, куда ссылается old_fd.
- int close(int fd); – освобождение файлового дескприптора.
- int execl(const char *filename, const char *arg1, …, NULL); – замена программы процесса.
- int usleep(useconds_t microsecs); – задержка программы на указанное количество микросекунд.
- pid_t waitpid(pid_t pid, int *wstatus, int options); – ожидание завершения дочернего процесса с id pid.
- int open(const char* filename, int flags, int access); – низкоуровневое открытие файла (возвращает файловый дескриптор).

Логика работы родительского процесса:

1. Настроить канал связи с будущим дочерним процессом.
2. Создать дочерний процесс и перенаправить потоки ввода и вывода в обоих процессах.
3. Запустить программу дочернего процесса и обработать команду пользователя.
4. Передать число в дочерний процесс и проверить завершенность дочернего процесса.
5. Продолжать выполнять пункт 4 до тех пор, пока дочерний процесс не завершит работу.
6. Обработать причину завершения дочернего процесса и вывести её в консоль.

Логика работы дочернего процесса:

1. Открыть файл и принять число от родителя.
2. Выполнить проверку на простоту.

3. В зависимости от исхода пункта 2 записать число в файл, либо завершится.

Код программы

parent.c

```
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>
#include <string.h>
#include <stdbool.h>

int checker(long long *res) {
    char buffer[32];

    errno = 0;

    ssize_t bytes_read = read(STDIN_FILENO, &buffer, sizeof(buffer) - 1);
    if (bytes_read < 0) {
        char message[] = "Error while reading.\n";
        write(STDERR_FILENO, message, sizeof(message));
        exit(EXIT_FAILURE);
    } else if (bytes_read == 0) {
        return -1;
    }

    buffer[bytes_read] = '\0';
    char *ptr;
    long long result = strtoll(buffer, &ptr, 10);

    if (buffer == ptr) {
        char message[] = "This is not a number, incorrect input.\n";
        write(STDERR_FILENO, message, sizeof(message));
        return 1;
    }

    if (!strcmp(ptr, "\n") && !strcmp(ptr, "\0")) { // \0 ставится, если это был конец файла
        char message[] = "There're more symbols after the number, incorrect input.\n";
        write(STDERR_FILENO, message, sizeof(message));
        return 1;
    }

    if (errno == ERANGE) {
        char message[] = "Too long number, incorrect input.\n";
        write(STDERR_FILENO, message, sizeof(message));
        return 1;
    }
```

```
    *res = result;
    return 0;
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        char message[] = "Usage: program_name filename\n";
        write(STDERR_FILENO, message, sizeof(message));
        return 1;
    }

    int parent_to_child[2];
    if (pipe(parent_to_child) == -1) {
        char message[] = "Error: failed tp spawn a pipe.\n";
        write(STDERR_FILENO, message, sizeof(message));
    }

    pid_t pid = fork();

    if (pid < 0) {
        char message[] = "Fork error.\n";
        write(STDERR_FILENO, message, sizeof(message));
        return 2;

    } else if (pid == 0) {
        dup2(parent_to_child[0], STDIN_FILENO);
        close(parent_to_child[0]);
        close(parent_to_child[1]);

        execl("./child", "child", argv[1], NULL);
        char message[] = "Error while executing the file: ";
        write(STDERR_FILENO, message, sizeof(message));
        write(STDERR_FILENO, argv[1], strlen(argv[1]));
        write(STDERR_FILENO, ".\n", sizeof(".\n"));
        return 2;

    } else {
        int stdout_backup = dup(STDOUT_FILENO);
        dup2(parent_to_child[1], STDOUT_FILENO);
        close(parent_to_child[0]);
        close(parent_to_child[1]);

        long long result = 0;
        int wstatus;
        pid_t flag = 0;

        while (true) {
            usleep(500);
            flag = waitpid(pid, &wstatus, WNOHANG);
```

```
    if (flag == -1) {
        char message[] = "Waitpid error.\n";
        write(STDERR_FILENO, &message, sizeof(message));
        return 2;
    } else if (flag > 0) {
        break;
    }

    int return_code = checker(&result);
    if (return_code == -1) {
        break;
    }

    else if (return_code == 0) {
        flag = waitpid(pid, &wstatus, WNOHANG);
        if (flag == -1) {
            char message[] = "Waitpid error.\n";
            write(STDERR_FILENO, &message, sizeof(message));
            return 2;
        } else if (flag > 0) {
            break;
        }
        write(STDOUT_FILENO, &result, sizeof(result));
    }
}

dup2(stdout_backup, STDOUT_FILENO);
close(stdout_backup);

if (flag == 0) {
    flag = waitpid(pid, &wstatus, 0);
}

if (flag == -1) {
    char message[] = "Waitpid error.\n";
    write(STDERR_FILENO, &message, sizeof(message));
    return 2;
}

if (WIFEXITED(wstatus)) {
    char message[] = "The decendant finished successfully.\n";
    write(STDOUT_FILENO, message, sizeof(message));
} else if (WIFSIGNALED(wstatus)) {
    char message[] = "The decendant was killed by signal.\n";
    write(STDOUT_FILENO, message, sizeof(message));
} else {
    char message[] = "Something went wrong.\n";
    write(STDOUT_FILENO, message, sizeof(message));
```

```
        }
    }
    return 0;
}
```

child.c

```
#include <unistd.h>
#include <math.h>
#include <fcntl.h>
#include <stdbool.h>

int main(int argc, char *argv[]) {
    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC | O_APPEND, 0600);
    if (fd == -1) {
        char message[] = "File opening error.\n";
        write(STDERR_FILENO, message, sizeof(message));
        return 1;
    }

    long long num;
    ssize_t res = 0;

    while (res = read(STDIN_FILENO, &num, sizeof(num))) {
        if (res < 0) {
            char message[] = "Error while reading.\n";
            write(STDERR_FILENO, message, sizeof(message));
            return 1; // ошибка обработки
        }

        if (num <= 3) {
            break;
        } else {
            if (num % 2 == 0 || num % 3 == 0) {
                ssize_t bytes = write(fd, &num, sizeof(num));

                if (bytes != sizeof(num)) {
                    char message[] = "File editing error.\n";
                    write(STDERR_FILENO, message, sizeof(message));
                    close(fd);
                    return 1;
                }
            } else {
                long long final = (long long)floor(sqrt((double)num)) + 1;
                bool fl = false;
                for (int i = 6; i <= final; i += 6) {
                    if (num % (i - 1) == 0 || num % (i + 1) == 0) {
                        ssize_t bytes = write(fd, &num, sizeof(num));
                        fl = true;
```

```
                        if (bytes != sizeof(num)) {
                            char message[] = "File editing error.\n";
                            write(STDERR_FILENO, message, sizeof(message));
                            close(fd);
                            return 1;
                        }
                        break;
                    }
                }
                if (fl == false) {
                    break;
                }
            }
        }
    }

    close(fd);

    return 0;
}
```

Протокол работы программы

Тесты:

1. 0 — процессы должны сразу завершится.
2. 1 — процессы должны сразу завершится.
3. 2 — процессы должны сразу завершится.
4. -1 — процессы должны сразу завершится.
5. Ctrl+D — процессы должны сразу завершится.
6. 6, 4, 2 — первые два числа записываются в файл, на третьем процессы завершаются.
7. 11111111111111111111111111111111, 0 — вывод ошибки слишком большего числа и продолжение программы.
8. 12аы, 0 — вывод ошибки вводда и продолжение программы.

Тестирование:

```
$ ./parent file.txt
0
The decendant finished successfully.
$ ./parent file.txt
1
The decendant finished successfully.
$ ./parent file.txt
2
The decendant finished successfully.
$ ./parent file.txt
-1
The decendant finished successfully.
$ ./parent file.txt
^D
The decendant finished successfully.
```

```
$ ./parent file.txt
6
4
2
The decendant finished successfully.
$ ./parent file.txt
11111111111111111111111111111111
0
The decendant finished successfully.
$ ./parent file.txt
12аы
0
The decendant finished successfully.
```

Вывод

На этой лабораторной работы были освоены навыки работы с процессами и установления связи между ними посредством каналов. В ходе работы я посмотрел на работу программ под другим углом.